In [1]:
```
import os
import nltk
#nltk.download()
```

In [2]:
```
import nltk.corpus
```

In [3]:
```
# you can also create your own words

AI = '''Artificial Intelligence refers to the intelligence of machines. This is in
humans and animals. With Artificial Intelligence, machines perform functions such a
problem-solving. Most noteworthy, Artificial Intelligence is the simulation of huma
It is probably the fastest-growing development in the World of technology and innov
AI could solve major challenges and crisis situations.'''
```

In [4]:
```
AI
```

Out[4]:
```
'Artificial Intelligence refers to the intelligence of machines. This is in contra
st to the natural intelligence of \nhumans and animals. With Artificial Intelligen
ce, machines perform functions such as learning, planning, reasoning and \nproblem
-solving. Most noteworthy, Artificial Intelligence is the simulation of human inte
lligence by machines. \nIt is probably the fastest-growing development in the Worl
d of technology and innovation. Furthermore, many experts believe\nAI could solve
major challenges and crisis situations.'
```

In [5]:
```
type(AI)
```

Out[5]:
```
str
```

In [11]:
```
from nltk.tokenize import word_tokenize
```

In [14]:
```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\LIKHITH\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\LIKHITH\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

Out[14]:
```
True
```

In [13]:
```
AI_tokens = word_tokenize(AI)
AI_tokens
```

Out[13]:
```
['Artificial',
 'Intelligence',
 'refers',
 'to',
 'the',
 'intelligence',
 'of',
 'machines',
 '.',
 'This',
 'is',
 'in',
 'contrast',
 'to',
 'the',
 'natural',
 'intelligence',
 'of',
 'humans',
 'and',
 'animals',
 '.',
 'With',
 'Artificial',
 'Intelligence',
 ',',
 'machines',
 'perform',
 'functions',
 'such',
 'as',
 'learning',
 ',',
 'planning',
 ',',
 'reasoning',
 'and',
 'problem-solving',
 '.',
 'Most',
 'noteworthy',
 ',',
 'Artificial',
 'Intelligence',
 'is',
 'the',
 'simulation',
 'of',
 'human',
 'intelligence',
 'by',
 'machines',
 '.',
 'It',
 'is',
 'probably',
```

```
 'the',
 'fastest-growing',
 'development',
 'in',
 'the',
 'World',
 'of',
 'technology',
 'and',
 'innovation',
 '.',
 'Furthermore',
 ',',
 'many',
 'experts',
 'believe',
 'AI',
 'could',
 'solve',
 'major',
 'challenges',
 'and',
 'crisis',
 'situations',
 '.']
```

In [15]:
```
len(AI_tokens)
```

Out[15]: 81

In [16]:
```
from nltk.tokenize import sent_tokenize
```

In [17]:
```
AI_sent = sent_tokenize(AI)
AI_sent
```

Out[17]:
```
['Artificial Intelligence refers to the intelligence of machines.',
 'This is in contrast to the natural intelligence of \nhumans and animals.',
 'With Artificial Intelligence, machines perform functions such as learning, plann
ing, reasoning and \nproblem-solving.',
 'Most noteworthy, Artificial Intelligence is the simulation of human intelligence
by machines.',
 'It is probably the fastest-growing development in the World of technology and in
novation.',
 'Furthermore, many experts believe\nAI could solve major challenges and crisis si
tuations.']
```

In [18]:
```
len(AI_sent)
```

Out[18]: 6

In [19]:
```
AI
```

Out[19]:
```
'Artificial Intelligence refers to the intelligence of machines. This is in contra
st to the natural intelligence of \nhumans and animals. With Artificial Intelligen
ce, machines perform functions such as learning, planning, reasoning and \nproblem
-solving. Most noteworthy, Artificial Intelligence is the simulation of human inte
lligence by machines. \nIt is probably the fastest-growing development in the Worl
d of technology and innovation. Furthermore, many experts believe\nAI could solve
major challenges and crisis situations.'
```

In [20]:
```
from nltk.tokenize import blankline_tokenize # GiVE YOU HOW MANY PARAGRAPH
AI_blank = blankline_tokenize(AI)
AI_blank
#AI_blank
```

Out[20]:
```
['Artificial Intelligence refers to the intelligence of machines. This is in contr
ast to the natural intelligence of \nhumans and animals. With Artificial Intellige
nce, machines perform functions such as learning, planning, reasoning and \nproble
m-solving. Most noteworthy, Artificial Intelligence is the simulation of human int
elligence by machines. \nIt is probably the fastest-growing development in the Wor
ld of technology and innovation. Furthermore, many experts believe\nAI could solve
major challenges and crisis situations.']
```

In [21]:
```
len(AI_blank)
```

Out[21]:
```
1
```

In [22]:
```
# NEXT WE WILL SEE HOW WE WILL USE UNI-GRAM,BI-GRAM,TRI-GRAM USING NLTK

from nltk.util import bigrams,trigrams,ngrams
```

In [23]:
```
string = 'the best and most beautifull thing in the world cannot be seen or even to
quotes_tokens = nltk.word_tokenize(string)
```

In [24]:
```
quotes_tokens
```

Out[24]:
```
['the',
 'best',
 'and',
 'most',
 'beautifull',
 'thing',
 'in',
 'the',
 'world',
 'can',
 'not',
 'be',
 'seen',
 'or',
 'even',
 'touched',
 ',',
 'they',
 'must',
 'be',
 'felt',
 'with',
 'heart']
```

In [25]:
```
len(quotes_tokens)
```

Out[25]:
```
23
```

In [26]:
```
quotes_bigrams = list(nltk.bigrams(quotes_tokens))
quotes_bigrams
```

Out[26]:
```
[('the', 'best'),
 ('best', 'and'),
 ('and', 'most'),
 ('most', 'beautifull'),
 ('beautifull', 'thing'),
 ('thing', 'in'),
 ('in', 'the'),
 ('the', 'world'),
 ('world', 'can'),
 ('can', 'not'),
 ('not', 'be'),
 ('be', 'seen'),
 ('seen', 'or'),
 ('or', 'even'),
 ('even', 'touched'),
 ('touched', ','),
 (',', 'they'),
 ('they', 'must'),
 ('must', 'be'),
 ('be', 'felt'),
 ('felt', 'with'),
 ('with', 'heart')]
```

In [27]:
```
quotes_tokens
```

Out[27]:
```
['the',
 'best',
 'and',
 'most',
 'beautifull',
 'thing',
 'in',
 'the',
 'world',
 'can',
 'not',
 'be',
 'seen',
 'or',
 'even',
 'touched',
 ',',
 'they',
 'must',
 'be',
 'felt',
 'with',
 'heart']
```

In [28]:
```
quotes_trigrams = list(nltk.trigrams(quotes_tokens))
quotes_trigrams
```

Out[28]:
```
[('the', 'best', 'and'),
 ('best', 'and', 'most'),
 ('and', 'most', 'beautifull'),
 ('most', 'beautifull', 'thing'),
 ('beautifull', 'thing', 'in'),
 ('thing', 'in', 'the'),
 ('in', 'the', 'world'),
 ('the', 'world', 'can'),
 ('world', 'can', 'not'),
 ('can', 'not', 'be'),
 ('not', 'be', 'seen'),
 ('be', 'seen', 'or'),
 ('seen', 'or', 'even'),
 ('or', 'even', 'touched'),
 ('even', 'touched', ','),
 ('touched', ',', 'they'),
 (',', 'they', 'must'),
 ('they', 'must', 'be'),
 ('must', 'be', 'felt'),
 ('be', 'felt', 'with'),
 ('felt', 'with', 'heart')]
```

In [29]:
```
quotes_trigrams = list(nltk.ngrams(quotes_tokens))
quotes_trigrams
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[29], line 1
----> 1 quotes_trigrams = list(nltk.ngrams(quotes_tokens))
      2 quotes_trigrams

TypeError: ngrams() missing 1 required positional argument: 'n'
```

In [30]:
```
quotes_ngrams = list(nltk.ngrams(quotes_tokens, 4))
quotes_ngrams

#it has given n-gram of length 4
```

Out[30]:
```
[('the', 'best', 'and', 'most'),
 ('best', 'and', 'most', 'beautifull'),
 ('and', 'most', 'beautifull', 'thing'),
 ('most', 'beautifull', 'thing', 'in'),
 ('beautifull', 'thing', 'in', 'the'),
 ('thing', 'in', 'the', 'world'),
 ('in', 'the', 'world', 'can'),
 ('the', 'world', 'can', 'not'),
 ('world', 'can', 'not', 'be'),
 ('can', 'not', 'be', 'seen'),
 ('not', 'be', 'seen', 'or'),
 ('be', 'seen', 'or', 'even'),
 ('seen', 'or', 'even', 'touched'),
 ('or', 'even', 'touched', ','),
 ('even', 'touched', ',', 'they'),
 ('touched', ',', 'they', 'must'),
 (',', 'they', 'must', 'be'),
 ('they', 'must', 'be', 'felt'),
 ('must', 'be', 'felt', 'with'),
 ('be', 'felt', 'with', 'heart')]
```

In [31]:
```
len(quotes_tokens)
```

Out[31]:
```
23
```

In [32]:
```
quotes_ngrams_1 = list(nltk.ngrams(quotes_tokens, 5))
quotes_ngrams_1
```

Out[32]:
```
[('the', 'best', 'and', 'most', 'beautifull'),
 ('best', 'and', 'most', 'beautifull', 'thing'),
 ('and', 'most', 'beautifull', 'thing', 'in'),
 ('most', 'beautifull', 'thing', 'in', 'the'),
 ('beautifull', 'thing', 'in', 'the', 'world'),
 ('thing', 'in', 'the', 'world', 'can'),
 ('in', 'the', 'world', 'can', 'not'),
 ('the', 'world', 'can', 'not', 'be'),
 ('world', 'can', 'not', 'be', 'seen'),
 ('can', 'not', 'be', 'seen', 'or'),
 ('not', 'be', 'seen', 'or', 'even'),
 ('be', 'seen', 'or', 'even', 'touched'),
 ('seen', 'or', 'even', 'touched', ','),
 ('or', 'even', 'touched', ',', 'they'),
 ('even', 'touched', ',', 'they', 'must'),
 ('touched', ',', 'they', 'must', 'be'),
 (',', 'they', 'must', 'be', 'felt'),
 ('they', 'must', 'be', 'felt', 'with'),
 ('must', 'be', 'felt', 'with', 'heart')]
```

In [33]:
```
quotes_ngrams = list(nltk.ngrams(quotes_tokens, 9))
quotes_ngrams
```

Out[33]:
```
[('the', 'best', 'and', 'most', 'beautifull', 'thing', 'in', 'the', 'world'),
 ('best', 'and', 'most', 'beautifull', 'thing', 'in', 'the', 'world', 'can'),
 ('and', 'most', 'beautifull', 'thing', 'in', 'the', 'world', 'can', 'not'),
 ('most', 'beautifull', 'thing', 'in', 'the', 'world', 'can', 'not', 'be'),
 ('beautifull', 'thing', 'in', 'the', 'world', 'can', 'not', 'be', 'seen'),
 ('thing', 'in', 'the', 'world', 'can', 'not', 'be', 'seen', 'or'),
 ('in', 'the', 'world', 'can', 'not', 'be', 'seen', 'or', 'even'),
 ('the', 'world', 'can', 'not', 'be', 'seen', 'or', 'even', 'touched'),
 ('world', 'can', 'not', 'be', 'seen', 'or', 'even', 'touched', ','),
 ('can', 'not', 'be', 'seen', 'or', 'even', 'touched', ',', 'they'),
 ('not', 'be', 'seen', 'or', 'even', 'touched', ',', 'they', 'must'),
 ('be', 'seen', 'or', 'even', 'touched', ',', 'they', 'must', 'be'),
 ('seen', 'or', 'even', 'touched', ',', 'they', 'must', 'be', 'felt'),
 ('or', 'even', 'touched', ',', 'they', 'must', 'be', 'felt', 'with'),
 ('even', 'touched', ',', 'they', 'must', 'be', 'felt', 'with', 'heart')]
```

In [34]:
```
from nltk.stem import PorterStemmer
pst = PorterStemmer()
```

In [35]:
```
pst.stem('having')
```

Out[35]: 'have'

In [36]:
```
pst.stem('affection')
```

Out[36]: 'affect'

In [37]:
```
pst.stem('playing')
```

Out[37]: 'play'

In [38]:
```
pst.stem('give')
```

Out[38]: 'give'

In [39]:
```
words_to_stem=['give','giving','given','gave']
for words in words_to_stem:
    print(words+  ':' + pst.stem(words))
```

```
give:give
giving:give
given:given
gave:gave
```

In [40]:
```
pst.stem('playing')
```

Out[40]: 'play'

In [41]:
```
words_to_stem=['give','giving','given','gave','thinking', 'loving', 'final', 'final
# i am giving these different words to stem, using porter stemmer we get the output

for words in words_to_stem:
    print(words+ ':' +pst.stem(words))

#in porterstemmer removes ing and replaces with e
```

```
give:give
giving:give
given:given
gave:gave
thinking:think
loving:love
final:final
finalized:final
finally:final
```

In [42]:
```
from nltk.stem import LancasterStemmer
lst = LancasterStemmer()
for words in words_to_stem:
    print(words + ':' + lst.stem(words))
```

```
give:giv
giving:giv
given:giv
gave:gav
thinking:think
loving:lov
final:fin
finalized:fin
finally:fin
```

In [43]:
```
words_to_stem=['give','giving','given','gave','thinking', 'loving', 'final', 'final
# i am giving these different words to stem, using porter stemmer we get the output

for words in words_to_stem:
    print(words+ ':' +pst.stem(words))
```

```
give:give
giving:give
given:given
gave:gave
thinking:think
loving:love
final:final
finalized:final
finally:final
```

In [44]:
```
from nltk.stem import SnowballStemmer
sbst = SnowballStemmer('english')
for words in words_to_stem:
    print(words+ ':' +sbst.stem(words))
```

```
give:give
giving:give
given:given
gave:gave
thinking:think
loving:love
final:final
finalized:final
finally:final
```

In [45]:
```
from nltk.stem import wordnet
from nltk.stem import WordNetLemmatizer
word_lem = WordNetLemmatizer()
```

In [46]:
```
words_to_stem
```

Out[46]:
```
['give',
 'giving',
 'given',
 'gave',
 'thinking',
 'loving',
 'final',
 'finalized',
 'finally']
```

In [47]:
```
for words in words_to_stem:
    print(words+ ':' +word_lem.lemmatize(words))
```

```
---------------------------------------------------------------------------
LookupError                               Traceback (most recent call last)
File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\util.py:84, in LazyCorpu
sLoader.__load(self)
     83 try:
---> 84     root = nltk.data.find(f"{self.subdir}/{zip_name}")
     85 except LookupError:

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\data.py:579, in find(resource_n
ame, paths)
    578 resource_not_found = f"\n{sep}\n{msg}\n{sep}\n"
--> 579 raise LookupError(resource_not_found)

LookupError:
**********************************************************************
  Resource wordnet not found.
  Please use the NLTK Downloader to obtain the resource:

  >>> import nltk
  >>> nltk.download('wordnet')

  For more information see: https://www.nltk.org/data.html

  Attempted to load corpora/wordnet.zip/wordnet/

  Searched in:
    - 'C:\\Users\\LIKHITH/nltk_data'
    - 'C:\\ProgramData\\anaconda3\\nltk_data'
    - 'C:\\ProgramData\\anaconda3\\share\\nltk_data'
    - 'C:\\ProgramData\\anaconda3\\lib\\nltk_data'
    - 'C:\\Users\\LIKHITH\\AppData\\Roaming\\nltk_data'
    - 'C:\\nltk_data'
    - 'D:\\nltk_data'
    - 'E:\\nltk_data'
**********************************************************************


During handling of the above exception, another exception occurred:

LookupError                               Traceback (most recent call last)
Cell In[47], line 2
      1 for words in words_to_stem:
----> 2     print(words+ ':' +word_lem.lemmatize(words))

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\stem\wordnet.py:85, in WordNetL
emmatizer.lemmatize(self, word, pos)
     60 def lemmatize(self, word: str, pos: str = "n") -> str:
     61     """Lemmatize `word` by picking the shortest of the possible lemmas,
     62     using the wordnet corpus reader's built-in _morphy function.
     63     Returns the input word unchanged if it cannot be found in WordNet.
   (...)
     83     :return: The shortest lemma of `word`, for the given `pos`.
     84     """
---> 85     lemmas = self._morphy(word, pos)
     86     return min(lemmas, key=len) if lemmas else word
```

```
File C:\ProgramData\anaconda3\Lib\site-packages\nltk\stem\wordnet.py:41, in WordNetL
emmatizer._morphy(self, form, pos, check_exceptions)
     31 """
     32 _morphy() is WordNet's _morphy lemmatizer.
     33 It returns a list of all lemmas found in WordNet.
   (...)
     37 ['us', 'u']
     38 """
     39 from nltk.corpus import wordnet as wn
---> 41 return wn._morphy(form, pos, check_exceptions)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\util.py:120, in LazyCorp
usLoader.__getattr__(self, attr)
    117 if attr == "__bases__":
    118     raise AttributeError("LazyCorpusLoader object has no attribute '__bases_
_'")
--> 120 self.__load()
    121 # This looks circular, but its not, since __load() changes our
    122 # __class__ to something new:
    123 return getattr(self, attr)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\util.py:86, in LazyCorpu
sLoader.__load(self)
     84             root = nltk.data.find(f"{self.subdir}/{zip_name}")
     85         except LookupError:
---> 86             raise e
     88 # Load the corpus.
     89 corpus = self.__reader_cls(root, *self.__args, **self.__kwargs)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\util.py:81, in LazyCorpu
sLoader.__load(self)
     79 else:
     80     try:
---> 81         root = nltk.data.find(f"{self.subdir}/{self.__name}")
     82     except LookupError as e:
     83         try:

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\data.py:579, in find(resource_n
ame, paths)
    577 sep = "*" * 70
    578 resource_not_found = f"\n{sep}\n{msg}\n{sep}\n"
--> 579 raise LookupError(resource_not_found)

LookupError:
**********************************************************************
  Resource wordnet not found.
  Please use the NLTK Downloader to obtain the resource:

  >>> import nltk
  >>> nltk.download('wordnet')

  For more information see: https://www.nltk.org/data.html

  Attempted to load corpora/wordnet

  Searched in:
```

```
    - 'C:\\Users\\LIKHITH/nltk_data'
    - 'C:\\ProgramData\\anaconda3\\nltk_data'
    - 'C:\\ProgramData\\anaconda3\\share\\nltk_data'
    - 'C:\\ProgramData\\anaconda3\\lib\\nltk_data'
    - 'C:\\Users\\LIKHITH\\AppData\\Roaming\\nltk_data'
    - 'C:\\nltk_data'
    - 'D:\\nltk_data'
    - 'E:\\nltk_data'
**********************************************************************
```

In [48]:
```
pst.stem('final')
```

Out[48]: 'final'

In [49]:
```
lst.stem('finally')
```

Out[49]: 'fin'

In [50]:
```
sbst.stem('finalized')
```

Out[50]: 'final'

In [51]:
```
lst.stem('final')
```

Out[51]: 'fin'

In [52]:
```
lst.stem('finalized')
```

Out[52]: 'fin'

In [55]:
```
from nltk.corpus import stopwords
```

In [57]:
```
import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\LIKHITH\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
```

Out[57]: True

In [58]:
```
stopwords.words('english')
```

Out[58]:
```
['a',
 'about',
 'above',
 'after',
 'again',
 'against',
 'ain',
 'all',
 'am',
 'an',
 'and',
 'any',
 'are',
 'aren',
 "aren't",
 'as',
 'at',
 'be',
 'because',
 'been',
 'before',
 'being',
 'below',
 'between',
 'both',
 'but',
 'by',
 'can',
 'couldn',
 "couldn't",
 'd',
 'did',
 'didn',
 "didn't",
 'do',
 'does',
 'doesn',
 "doesn't",
 'doing',
 'don',
 "don't",
 'down',
 'during',
 'each',
 'few',
 'for',
 'from',
 'further',
 'had',
 'hadn',
 "hadn't",
 'has',
 'hasn',
 "hasn't",
 'have',
 'haven',
```

```
"haven't",
'having',
'he',
"he'd",
"he'll",
'her',
'here',
'hers',
'herself',
"he's",
'him',
'himself',
'his',
'how',
'i',
"i'd",
'if',
"i'll",
"i'm",
'in',
'into',
'is',
'isn',
"isn't",
'it',
"it'd",
"it'll",
"it's",
'its',
'itself',
"i've",
'just',
'll',
'm',
'ma',
'me',
'mightn',
"mightn't",
'more',
'most',
'mustn',
"mustn't",
'my',
'myself',
'needn',
"needn't",
'no',
'nor',
'not',
'now',
'o',
'of',
'off',
'on',
'once',
'only',
```

```
'or',
'other',
'our',
'ours',
'ourselves',
'out',
'over',
'own',
're',
's',
'same',
'shan',
"shan't",
'she',
"she'd",
"she'll",
"she's",
'should',
'shouldn',
"shouldn't",
"should've",
'so',
'some',
'such',
't',
'than',
'that',
"that'll",
'the',
'their',
'theirs',
'them',
'themselves',
'then',
'there',
'these',
'they',
"they'd",
"they'll",
"they're",
"they've",
'this',
'those',
'through',
'to',
'too',
'under',
'until',
'up',
've',
'very',
'was',
'wasn',
"wasn't",
'we',
"we'd",
```

```
 "we'll",
 "we're",
 'were',
 'weren',
 "weren't",
 "we've",
 'what',
 'when',
 'where',
 'which',
 'while',
 'who',
 'whom',
 'why',
 'will',
 'with',
 'won',
 "won't",
 'wouldn',
 "wouldn't",
 'y',
 'you',
 "you'd",
 "you'll",
 'your',
 "you're",
 'yours',
 'yourself',
 'yourselves',
 "you've"]
```

In [59]:
```
len(stopwords.words('english'))
```

Out[59]: 198

In [60]:
```
stopwords.words('spanish')
```

Out[60]:
```
['de',
 'la',
 'que',
 'el',
 'en',
 'y',
 'a',
 'los',
 'del',
 'se',
 'las',
 'por',
 'un',
 'para',
 'con',
 'no',
 'una',
 'su',
 'al',
 'lo',
 'como',
 'más',
 'pero',
 'sus',
 'le',
 'ya',
 'o',
 'este',
 'sí',
 'porque',
 'esta',
 'entre',
 'cuando',
 'muy',
 'sin',
 'sobre',
 'también',
 'me',
 'hasta',
 'hay',
 'donde',
 'quien',
 'desde',
 'todo',
 'nos',
 'durante',
 'todos',
 'uno',
 'les',
 'ni',
 'contra',
 'otros',
 'ese',
 'eso',
 'ante',
 'ellos',
```

```
'e',
'esto',
'mí',
'antes',
'algunos',
'qué',
'unos',
'yo',
'otro',
'otras',
'otra',
'él',
'tanto',
'esa',
'estos',
'mucho',
'quienes',
'nada',
'muchos',
'cual',
'poco',
'ella',
'estar',
'estas',
'algunas',
'algo',
'nosotros',
'mi',
'mis',
'tú',
'te',
'ti',
'tu',
'tus',
'ellas',
'nosotras',
'vosotros',
'vosotras',
'os',
'mío',
'mía',
'míos',
'mías',
'tuyo',
'tuya',
'tuyos',
'tuyas',
'suyo',
'suya',
'suyos',
'suyas',
'nuestro',
'nuestra',
'nuestros',
'nuestras',
'vuestro',
```

```
                'vuestra',
                'vuestros',
                'vuestras',
                'esos',
                'esas',
                'estoy',
                'estás',
                'está',
                'estamos',
                'estáis',
                'están',
                'esté',
                'estés',
                'estemos',
                'estéis',
                'estén',
                'estaré',
                'estarás',
                'estará',
                'estaremos',
                'estaréis',
                'estarán',
                'estaría',
                'estarías',
                'estaríamos',
                'estaríais',
                'estarían',
                'estaba',
                'estabas',
                'estábamos',
                'estabais',
                'estaban',
                'estuve',
                'estuviste',
                'estuvo',
                'estuvimos',
                'estuvisteis',
                'estuvieron',
                'estuviera',
                'estuvieras',
                'estuviéramos',
                'estuvierais',
                'estuvieran',
                'estuviese',
                'estuvieses',
                'estuviésemos',
                'estuvieseis',
                'estuviesen',
                'estando',
                'estado',
                'estada',
                'estados',
                'estadas',
                'estad',
                'he',
                'has',
```

```
'ha',
'hemos',
'habéis',
'han',
'haya',
'hayas',
'hayamos',
'hayáis',
'hayan',
'habré',
'habrás',
'habrá',
'habremos',
'habréis',
'habrán',
'habría',
'habrías',
'habríamos',
'habríais',
'habrían',
'había',
'habías',
'habíamos',
'habíais',
'habían',
'hube',
'hubiste',
'hubo',
'hubimos',
'hubisteis',
'hubieron',
'hubiera',
'hubieras',
'hubiéramos',
'hubierais',
'hubieran',
'hubiese',
'hubieses',
'hubiésemos',
'hubieseis',
'hubiesen',
'habiendo',
'habido',
'habida',
'habidos',
'habidas',
'soy',
'eres',
'es',
'somos',
'sois',
'son',
'sea',
'seas',
'seamos',
'seáis',
```

```
                'sean',
                'seré',
                'serás',
                'será',
                'seremos',
                'seréis',
                'serán',
                'sería',
                'serías',
                'seríamos',
                'seríais',
                'serían',
                'era',
                'eras',
                'éramos',
                'erais',
                'eran',
                'fui',
                'fuiste',
                'fue',
                'fuimos',
                'fuisteis',
                'fueron',
                'fuera',
                'fueras',
                'fuéramos',
                'fuerais',
                'fueran',
                'fuese',
                'fueses',
                'fuésemos',
                'fueseis',
                'fuesen',
                'sintiendo',
                'sentido',
                'sentida',
                'sentidos',
                'sentidas',
                'siente',
                'sentid',
                'tengo',
                'tienes',
                'tiene',
                'tenemos',
                'tenéis',
                'tienen',
                'tenga',
                'tengas',
                'tengamos',
                'tengáis',
                'tengan',
                'tendré',
                'tendrás',
                'tendrá',
                'tendremos',
                'tendréis',
```

```
 'tendrán',
 'tendría',
 'tendrías',
 'tendríamos',
 'tendríais',
 'tendrían',
 'tenía',
 'tenías',
 'teníamos',
 'teníais',
 'tenían',
 'tuve',
 'tuviste',
 'tuvo',
 'tuvimos',
 'tuvisteis',
 'tuvieron',
 'tuviera',
 'tuvieras',
 'tuviéramos',
 'tuvierais',
 'tuvieran',
 'tuviese',
 'tuvieses',
 'tuviésemos',
 'tuvieseis',
 'tuviesen',
 'teniendo',
 'tenido',
 'tenida',
 'tenidos',
 'tenidas',
 'tened']
```

In [61]:
```
len(stopwords.words('spanish'))
```

Out[61]: 313

In [62]:
```
stopwords.words('french')
```

Out[62]:
```
['au',
 'aux',
 'avec',
 'ce',
 'ces',
 'dans',
 'de',
 'des',
 'du',
 'elle',
 'en',
 'et',
 'eux',
 'il',
 'ils',
 'je',
 'la',
 'le',
 'les',
 'leur',
 'lui',
 'ma',
 'mais',
 'me',
 'même',
 'mes',
 'moi',
 'mon',
 'ne',
 'nos',
 'notre',
 'nous',
 'on',
 'ou',
 'par',
 'pas',
 'pour',
 'qu',
 'que',
 'qui',
 'sa',
 'se',
 'ses',
 'son',
 'sur',
 'ta',
 'te',
 'tes',
 'toi',
 'ton',
 'tu',
 'un',
 'une',
 'vos',
 'votre',
 'vous',
```

```
'c',
'd',
'j',
'l',
'à',
'm',
'n',
's',
't',
'y',
'été',
'étée',
'étées',
'étés',
'étant',
'étante',
'étants',
'étantes',
'suis',
'es',
'est',
'sommes',
'êtes',
'sont',
'serai',
'seras',
'sera',
'serons',
'serez',
'seront',
'serais',
'serait',
'serions',
'seriez',
'seraient',
'étais',
'était',
'étions',
'étiez',
'étaient',
'fus',
'fut',
'fûmes',
'fûtes',
'furent',
'sois',
'soit',
'soyons',
'soyez',
'soient',
'fusse',
'fusses',
'fût',
'fussions',
'fussiez',
'fussent',
```

```
 'ayant',
 'ayante',
 'ayantes',
 'ayants',
 'eu',
 'eue',
 'eues',
 'eus',
 'ai',
 'as',
 'avons',
 'avez',
 'ont',
 'aurai',
 'auras',
 'aura',
 'aurons',
 'aurez',
 'auront',
 'aurais',
 'aurait',
 'aurions',
 'auriez',
 'auraient',
 'avais',
 'avait',
 'avions',
 'aviez',
 'avaient',
 'eut',
 'eûmes',
 'eûtes',
 'eurent',
 'aie',
 'aies',
 'ait',
 'ayons',
 'ayez',
 'aient',
 'eusse',
 'eusses',
 'eût',
 'eussions',
 'eussiez',
 'eussent']
```

In [63]:
```
stopwords.words('german')
```

Out[63]:
```
['aber',
 'alle',
 'allem',
 'allen',
 'aller',
 'alles',
 'als',
 'also',
 'am',
 'an',
 'ander',
 'andere',
 'anderem',
 'anderen',
 'anderer',
 'anderes',
 'anderm',
 'andern',
 'anderr',
 'anders',
 'auch',
 'auf',
 'aus',
 'bei',
 'bin',
 'bis',
 'bist',
 'da',
 'damit',
 'dann',
 'der',
 'den',
 'des',
 'dem',
 'die',
 'das',
 'dass',
 'daß',
 'derselbe',
 'derselben',
 'denselben',
 'desselben',
 'demselben',
 'dieselbe',
 'dieselben',
 'dasselbe',
 'dazu',
 'dein',
 'deine',
 'deinem',
 'deinen',
 'deiner',
 'deines',
 'denn',
 'derer',
 'dessen',
```

```
                'dich',
                'dir',
                'du',
                'dies',
                'diese',
                'diesem',
                'diesen',
                'dieser',
                'dieses',
                'doch',
                'dort',
                'durch',
                'ein',
                'eine',
                'einem',
                'einen',
                'einer',
                'eines',
                'einig',
                'einige',
                'einigem',
                'einigen',
                'einiger',
                'einiges',
                'einmal',
                'er',
                'ihn',
                'ihm',
                'es',
                'etwas',
                'euer',
                'eure',
                'eurem',
                'euren',
                'eurer',
                'eures',
                'für',
                'gegen',
                'gewesen',
                'hab',
                'habe',
                'haben',
                'hat',
                'hatte',
                'hatten',
                'hier',
                'hin',
                'hinter',
                'ich',
                'mich',
                'mir',
                'ihr',
                'ihre',
                'ihrem',
                'ihren',
                'ihrer',
```

```
                    'ihres',
                    'euch',
                    'im',
                    'in',
                    'indem',
                    'ins',
                    'ist',
                    'jede',
                    'jedem',
                    'jeden',
                    'jeder',
                    'jedes',
                    'jene',
                    'jenem',
                    'jenen',
                    'jener',
                    'jenes',
                    'jetzt',
                    'kann',
                    'kein',
                    'keine',
                    'keinem',
                    'keinen',
                    'keiner',
                    'keines',
                    'können',
                    'könnte',
                    'machen',
                    'man',
                    'manche',
                    'manchem',
                    'manchen',
                    'mancher',
                    'manches',
                    'mein',
                    'meine',
                    'meinem',
                    'meinen',
                    'meiner',
                    'meines',
                    'mit',
                    'muss',
                    'musste',
                    'nach',
                    'nicht',
                    'nichts',
                    'noch',
                    'nun',
                    'nur',
                    'ob',
                    'oder',
                    'ohne',
                    'sehr',
                    'sein',
                    'seine',
                    'seinem',
```

```
                'seinen',
                'seiner',
                'seines',
                'selbst',
                'sich',
                'sie',
                'ihnen',
                'sind',
                'so',
                'solche',
                'solchem',
                'solchen',
                'solcher',
                'solches',
                'soll',
                'sollte',
                'sondern',
                'sonst',
                'über',
                'um',
                'und',
                'uns',
                'unsere',
                'unserem',
                'unseren',
                'unser',
                'unseres',
                'unter',
                'viel',
                'vom',
                'von',
                'vor',
                'während',
                'war',
                'waren',
                'warst',
                'was',
                'weg',
                'weil',
                'weiter',
                'welche',
                'welchem',
                'welchen',
                'welcher',
                'welches',
                'wenn',
                'werde',
                'werden',
                'wie',
                'wieder',
                'will',
                'wir',
                'wird',
                'wirst',
                'wo',
                'wollen',
```

```
 'wollte',
 'würde',
 'würden',
 'zu',
 'zum',
 'zur',
 'zwar',
 'zwischen']
```

In [64]:
```
len(stopwords.words('german'))
```

Out[64]: 232

In [65]:
```
stopwords.words('hindi') # research phase
```

```
---------------------------------------------------------------------------
OSError                                   Traceback (most recent call last)
Cell In[65], line 1
----> 1 stopwords.words('hindi')

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\reader\wordlist.py:21, i
n WordListCorpusReader.words(self, fileids, ignore_lines_startswith)
     18 def words(self, fileids=None, ignore_lines_startswith="\n"):
     19     return [
     20         line
---> 21         for line in line_tokenize(self.raw(fileids))
     22         if not line.startswith(ignore_lines_startswith)
     23     ]

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\reader\api.py:218, in Co
rpusReader.raw(self, fileids)
    216 contents = []
    217 for f in fileids:
--> 218     with self.open(f) as fp:
    219         contents.append(fp.read())
    220 return concat(contents)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\corpus\reader\api.py:231, in Co
rpusReader.open(self, file)
    223 """
    224 Return an open stream that can be used to read the given file.
    225 If the file's encoding is not None, then the stream will
   (...)
    228 :param file: The file identifier of the file to read.
    229 """
    230 encoding = self.encoding(file)
--> 231 stream = self._root.join(file).open(encoding)
    232 return stream

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\data.py:333, in FileSystemPathP
ointer.join(self, fileid)
    331 def join(self, fileid):
    332     _path = os.path.join(self._path, fileid)
--> 333     return FileSystemPathPointer(_path)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\data.py:311, in FileSystemPathP
ointer.__init__(self, _path)
    309 _path = os.path.abspath(_path)
    310 if not os.path.exists(_path):
--> 311     raise OSError("No such file or directory: %r" % _path)
    312 self._path = _path

OSError: No such file or directory: 'C:\\Users\\LIKHITH\\AppData\\Roaming\\nltk_data
\\corpora\\stopwords\\hindi'
```

In [66]:
```
import re
punctuation = re.compile(r'[-.?!,:;()|0-9]')
```

In [67]:
```
punctuation
```

Out[67]:
```
re.compile(r'[-.?!,:;()|0-9]', re.UNICODE)
```

In [68]:
```
AI
```

Out[68]:
```
'Artificial Intelligence refers to the intelligence of machines. This is in contra
st to the natural intelligence of \nhumans and animals. With Artificial Intelligen
ce, machines perform functions such as learning, planning, reasoning and \nproblem
-solving. Most noteworthy, Artificial Intelligence is the simulation of human inte
lligence by machines. \nIt is probably the fastest-growing development in the Worl
d of technology and innovation. Furthermore, many experts believe\nAI could solve
major challenges and crisis situations.'
```

In [69]:
```
AI_tokens
```

Out[69]:
```
['Artificial',
 'Intelligence',
 'refers',
 'to',
 'the',
 'intelligence',
 'of',
 'machines',
 '.',
 'This',
 'is',
 'in',
 'contrast',
 'to',
 'the',
 'natural',
 'intelligence',
 'of',
 'humans',
 'and',
 'animals',
 '.',
 'With',
 'Artificial',
 'Intelligence',
 ',',
 'machines',
 'perform',
 'functions',
 'such',
 'as',
 'learning',
 ',',
 'planning',
 ',',
 'reasoning',
 'and',
 'problem-solving',
 '.',
 'Most',
 'noteworthy',
 ',',
 'Artificial',
 'Intelligence',
 'is',
 'the',
 'simulation',
 'of',
 'human',
 'intelligence',
 'by',
 'machines',
 '.',
 'It',
 'is',
 'probably',
```

```
 'the',
 'fastest-growing',
 'development',
 'in',
 'the',
 'World',
 'of',
 'technology',
 'and',
 'innovation',
 '.',
 'Furthermore',
 ',',
 'many',
 'experts',
 'believe',
 'AI',
 'could',
 'solve',
 'major',
 'challenges',
 'and',
 'crisis',
 'situations',
 '.']
```

In [70]:
```
len(AI_tokens)
```

Out[70]: 81

In [71]:
```
    #POS [part of sppech] is always talking about grammaticaly type of the word cal

#how the word will function in grammatically within the sentence, a word can have m

#so lets see some pos tags & description, so pos tags are usualy used to descrie we

#in this slide we have so many tags along with their description with different tag

#this tags are beginning from coordinating conjunction to whadverb & lets understan

#next we will see how we will implement this POS in our text
```

In [72]:
```
# we will see how to work in POS using NLTK library

sent = 'kathy is a natural when it comes to drawing'
sent_tokens = word_tokenize(sent)
sent_tokens

# first we will tokenize usning word_tokenize & then we will use pos_tag on all of
```

Out[72]: ['kathy', 'is', 'a', 'natural', 'when', 'it', 'comes', 'to', 'drawing']

In [73]:
```
for token in sent_tokens:
    print(nltk.pos_tag([token]))
```

```
---------------------------------------------------------------------------
LookupError                               Traceback (most recent call last)
Cell In[73], line 2
      1 for token in sent_tokens:
----> 2     print(nltk.pos_tag([token]))

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\tag\__init__.py:168, in pos_tag
(tokens, tagset, lang)
    143 def pos_tag(tokens, tagset=None, lang="eng"):
    144     """
    145     Use NLTK's currently recommended part of speech tagger to
    146     tag the given list of tokens.
   (...)
    166     :rtype: list(tuple(str, str))
    167     """
--> 168     tagger = _get_tagger(lang)
    169     return _pos_tag(tokens, tagset, tagger, lang)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\tag\__init__.py:110, in _get_ta
gger(lang)
    108     tagger = PerceptronTagger(lang=lang)
    109 else:
--> 110     tagger = PerceptronTagger()
    111 return tagger

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\tag\perceptron.py:183, in Perce
ptronTagger.__init__(self, load, lang)
    181 self.classes = set()
    182 if load:
--> 183     self.load_from_json(lang)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\tag\perceptron.py:273, in Perce
ptronTagger.load_from_json(self, lang)
    271 def load_from_json(self, lang="eng"):
    272     # Automatically find path to the tagger if location is not specified.
--> 273     loc = find(f"taggers/averaged_perceptron_tagger_{lang}/")
    274     with open(loc + TAGGER_JSONS[lang]["weights"]) as fin:
    275         self.model.weights = json.load(fin)

File C:\ProgramData\anaconda3\Lib\site-packages\nltk\data.py:579, in find(resource_n
ame, paths)
    577 sep = "*" * 70
    578 resource_not_found = f"\n{sep}\n{msg}\n{sep}\n"
--> 579 raise LookupError(resource_not_found)

LookupError:
**********************************************************************
  Resource averaged_perceptron_tagger_eng not found.
  Please use the NLTK Downloader to obtain the resource:

  >>> import nltk
  >>> nltk.download('averaged_perceptron_tagger_eng')

  For more information see: https://www.nltk.org/data.html

  Attempted to load taggers/averaged_perceptron_tagger_eng/
```

```
  Searched in:
    - 'C:\\Users\\LIKHITH/nltk_data'
    - 'C:\\ProgramData\\anaconda3\\nltk_data'
    - 'C:\\ProgramData\\anaconda3\\share\\nltk_data'
    - 'C:\\ProgramData\\anaconda3\\lib\\nltk_data'
    - 'C:\\Users\\LIKHITH\\AppData\\Roaming\\nltk_data'
    - 'C:\\nltk_data'
    - 'D:\\nltk_data'
    - 'E:\\nltk_data'
**********************************************************************
```

In [74]:
```
import nltk
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\LIKHITH\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

Out[74]: True

In [75]:
```
for token in sent_tokens:
    print(nltk.pos_tag([token]))
```

```
[('kathy', 'NN')]
[('is', 'VBZ')]
[('a', 'DT')]
[('natural', 'JJ')]
[('when', 'WRB')]
[('it', 'PRP')]
[('comes', 'VBZ')]
[('to', 'TO')]
[('drawing', 'VBG')]
```

In [76]:
```
sent2 = 'john is eating a delicious cake'
sent2_tokens = word_tokenize(sent2)

for token in sent2_tokens:
    print(nltk.pos_tag([token]))
```

```
[('john', 'NN')]
[('is', 'VBZ')]
[('eating', 'VBG')]
[('a', 'DT')]
[('delicious', 'JJ')]
[('cake', 'NN')]
```

In [77]:
```
from nltk import ne_chunk
```

In [78]:
```
NE_sent = 'The US president stays in the WHITEHOUSE '
```

In [79]:
```
NE_tokens = word_tokenize(NE_sent)

#after tokenize need to add the pos tags
NE_tokens
```

Out[79]:
```
['The', 'US', 'president', 'stays', 'in', 'the', 'WHITEHOUSE']
```

In [80]:
```
NE_tags = nltk.pos_tag(NE_tokens)
NE_tags
```

Out[80]:
```
[('The', 'DT'),
 ('US', 'NNP'),
 ('president', 'NN'),
 ('stays', 'NNS'),
 ('in', 'IN'),
 ('the', 'DT'),
 ('WHITEHOUSE', 'NNP')]
```

In [86]:
```
import nltk
nltk.download('maxent_ne_chunker_tab')
```

```
[nltk_data] Downloading package maxent_ne_chunker_tab to
[nltk_data]     C:\Users\LIKHITH\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping chunkers\maxent_ne_chunker_tab.zip.
```

Out[86]:
```
True
```

In [87]:
```
#we are passin the NE_NER into ne_chunks function and lets see the outputs

NE_NER = ne_chunk(NE_tags)
print(NE_NER)
```

```
(S
  The/DT
  (GSP US/NNP)
  president/NN
  stays/NNS
  in/IN
  the/DT
  (ORGANIZATION WHITEHOUSE/NNP))
```

In [88]:
```
new = 'the big cat ate the little mouse who was after fresh cheese'
new_tokens = nltk.pos_tag(word_tokenize(new))
new_tokens
```

Out[88]:
```
[('the', 'DT'),
 ('big', 'JJ'),
 ('cat', 'NN'),
 ('ate', 'VBD'),
 ('the', 'DT'),
 ('little', 'JJ'),
 ('mouse', 'NN'),
 ('who', 'WP'),
 ('was', 'VBD'),
 ('after', 'IN'),
 ('fresh', 'JJ'),
 ('cheese', 'NN')]
```

In [89]:
```
# Libraries
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[89], line 2
      1 # Libraries
----> 2 from wordcloud import WordCloud
      3 import matplotlib.pyplot as plt

ModuleNotFoundError: No module named 'wordcloud'
```

In [90]:
```
pip install wordcloud
```

```
Defaulting to user installation because normal site-packages is not writeable
Collecting wordcloud
  Downloading wordcloud-1.9.6-cp313-cp313-win_amd64.whl.metadata (3.5 kB)
Requirement already satisfied: numpy>=1.19.3 in C:\ProgramData\anaconda3\Lib\site-pa
ckages (from wordcloud) (2.1.3)
Requirement already satisfied: pillow in C:\ProgramData\anaconda3\Lib\site-packages
(from wordcloud) (11.1.0)
Requirement already satisfied: matplotlib in C:\ProgramData\anaconda3\Lib\site-packa
ges (from wordcloud) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in C:\ProgramData\anaconda3\Lib\site
-packages (from matplotlib->wordcloud) (1.3.1)
Requirement already satisfied: cycler>=0.10 in C:\ProgramData\anaconda3\Lib\site-pac
kages (from matplotlib->wordcloud) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in C:\ProgramData\anaconda3\Lib\sit
e-packages (from matplotlib->wordcloud) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in C:\ProgramData\anaconda3\Lib\sit
e-packages (from matplotlib->wordcloud) (1.4.8)
Requirement already satisfied: packaging>=20.0 in C:\ProgramData\anaconda3\Lib\site-
packages (from matplotlib->wordcloud) (24.2)
Requirement already satisfied: pyparsing>=2.3.1 in C:\ProgramData\anaconda3\Lib\site
-packages (from matplotlib->wordcloud) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in C:\ProgramData\anaconda3\Lib
\site-packages (from matplotlib->wordcloud) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in C:\ProgramData\anaconda3\Lib\site-package
s (from python-dateutil>=2.7->matplotlib->wordcloud) (1.17.0)
Downloading wordcloud-1.9.6-cp313-cp313-win_amd64.whl (306 kB)
Installing collected packages: wordcloud
Successfully installed wordcloud-1.9.6
Note: you may need to restart the kernel to use updated packages.
  WARNING: The script wordcloud_cli.exe is installed in 'C:\Users\LIKHITH\AppData\Ro
aming\Python\Python313\Scripts' which is not on PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning,
use --no-warn-script-location.

[notice] A new release of pip is available: 26.0 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

In [91]:
```
# Libraries
from wordcloud import WordCloud
import matplotlib.pyplot as plt
```

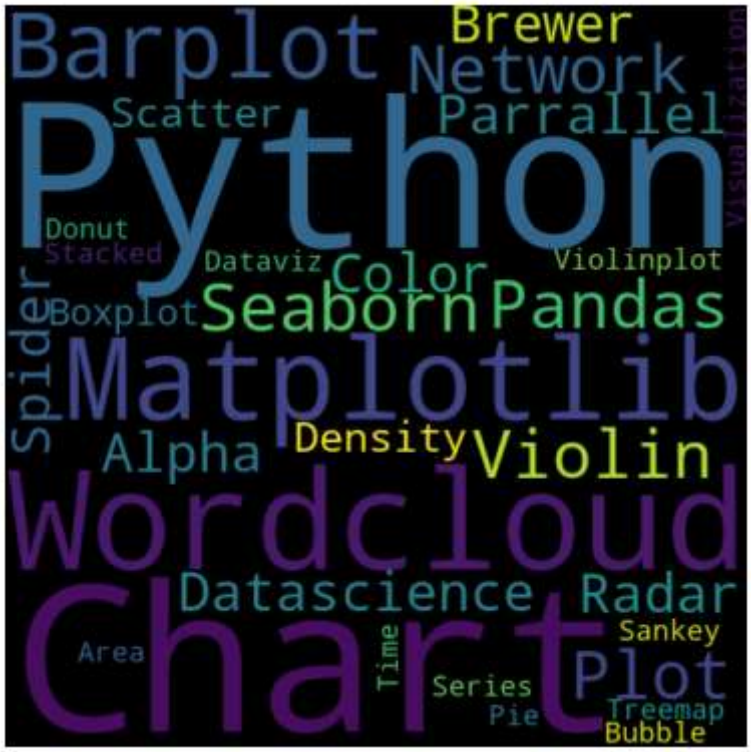
In [92]:
```
# Create a list of word
text=("Python Python Python Matplotlib Matplotlib Seaborn Network Plot Violin Chart
```

In [93]:
```
text
```

Out[93]:
```
'Python Python Python Matplotlib Matplotlib Seaborn Network Plot Violin Chart Pand
as Datascience Wordcloud Spider Radar Parrallel Alpha Color Brewer Density Scatter
Barplot Barplot Boxplot Violinplot Treemap Stacked Area Chart Chart Visualization
Dataviz Donut Pie Time-Series Wordcloud Wordcloud Sankey Bubble'
```

In [94]:
```
# Create the wordcloud object
wordcloud = WordCloud(width=480, height=480, margin=0).generate(text)
```

In [95]:
```
# Display the generated image:
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis("off")
plt.margins(x=0, y=0)
plt.show()
```



In [ ]:
```

```